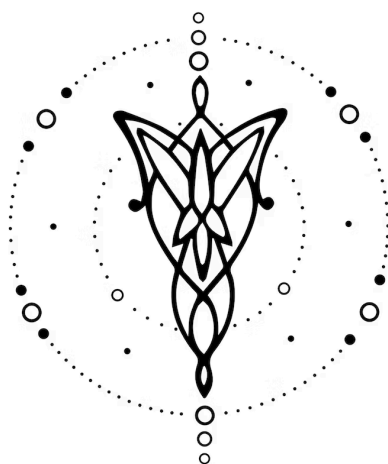




Entrega Final

DISEÑO E IMPLEMENTACIÓN DE UN LENGUAJE

v1.0.0

1. Equipo	1
2. Repositorio	2
3. Introduccion	2
4. Modelo Computacional	4
5. Implementacion	7
6. Futuras Extensiones	8
7. Conclusiones	8
8. Referencias	8
9. Bibliografía	8

1. Equipo

Nombre	Apellido	Legajo	E-mail
Lucila	Borinsky	63039	lborkinsky@itba.edu.ar
Santiago Diaz	Sieiro	63058	sdiazsieiro@itba.edu.ar
Axel Armando	Farina	63081	axfarina@itba.edu.ar
Hwa Pyoung	Kim	62129	hkim@itba.edu.ar

2. Repositorio

La solución y su documentación serán versionadas en: [TLA PLOTTER](#).

3. Introducción

La generación de gráficos vectoriales programáticos es una necesidad común en el desarrollo de software moderno, abarcando desde la creación de diagramas dinámicos hasta la visualización de datos en tiempo real. El estándar de facto para gráficos vectoriales en la web es **SVG** (Scalable Vector Graphics), un formato basado en XML.

Sin embargo, escribir SVG directamente presenta desafíos significativos: su sintaxis es verbosa, propensa a errores debido a la estricta jerarquía XML y carece de abstracciones de alto nivel como variables, cálculos matemáticos o reutilización modular de componentes con parámetros dinámicos.

Con el objetivo de resolver estas limitaciones y simplificar drásticamente la creación de escenas gráficas 2D, presentamos **Plotter**. Este proyecto consiste en el diseño e implementación de un Lenguaje de Dominio Específico (DSL) declarativo, junto con su correspondiente compilador.

Plotter permite a los usuarios describir escenas complejas utilizando una sintaxis limpia, inspirada en lenguajes de configuración modernos y CSS, abstrae la complejidad del sistema de coordenadas y ofrece herramientas potentes como capas (layers), grupos (groups), símbolos reutilizables (symbols) y paletas de colores semánticas. El compilador se encarga de traducir estas descripciones de alto nivel en código SVG optimizado y válido, listo para ser renderizado en cualquier navegador.

4. Modelo Computacional

4.1. Dominio del Problema

El dominio abordado es la **Gráfica Vectorial 2D Descriptiva**. A diferencia de los gráficos rasterizados (mapas de bits), los gráficos vectoriales se definen mediante primitivas geométricas (puntos, líneas, curvas y polígonos) basadas en fórmulas matemáticas. Esto permite que las imágenes sean escalables infinitamente sin pérdida de calidad, una propiedad fundamental para interfaces de usuario responsivas y diagramas técnicos.

El modelo computacional de **Plotter** distingue claramente entre dos espacios:

1. **Espacio de Definición Lógica:** Donde el usuario describe qué elementos componen la escena, sus propiedades y relaciones jerárquicas, sin preocuparse necesariamente por el orden de escritura.

2. **Espacio de Renderizado Visual:** El resultado final donde el orden de superposición (Z-order) es crítico. **Plotter** introduce el concepto explícito de Capas (Layers) con índices Z (z-index) para gestionar la profundidad visual de manera intuitiva, algo que SVG maneja implícitamente sólo por orden de aparición.

4.2. Características del Lenguaje

El lenguaje diseñado ofrece un conjunto rico de abstracciones:

4.2.1 Estructura General

Todo programa comienza con la definición de una scene, que actúa como el lienzo principal. Dentro de ella, se pueden declarar recursos globales (paletas, símbolos) y elementos visuales.

Ejemplo de estructura básica:

```
// Escena mínima
scene MiPaisaje {

  // Recursos
  palette misColores {...}

  // Capas
  layer fondo z 0 {...}
  layer frente z 10 {...}
```

4.2.2 Primitivas Geométricas

Soporte nativo para las figuras fundamentales:

- *rectangle*, *circle*, *ellipse*: Formas básicas con propiedades de dimensión y radio.
- *line*, *polyline*, *polygon*: Figuras basadas en vértices para formas complejas.

4.2.3 Sistema de Estilos y Unidades

Plotter soporta un sistema de unidades flexible (*px*, *rem*, *em*, *%*, *vw*, *vh*) que se traduce directamente a SVG. Los colores pueden definirse en múltiples formatos:

- Hexadecimal: `#FF5733`
- Funciones: `rgb(255, 87, 51)`, `rgba(255, 87, 51, 0.8)`
- Referencias Semánticas: `miPaleta.principal`

4.2.4 Transformaciones y Agrupamiento

Se permite agrupar elementos lógicamente mediante *group* y aplicar transformaciones afines (*translate*, *rotate*, *scale*) tanto a figuras individuales como a grupos completos o instancias de símbolos.

4.2.5 Variables y Expresiones

Una de las características más potentes es la capacidad de usar variables y expresiones aritméticas en las propiedades. Esto permite definir relaciones dinámicas, como centrar un objeto ($x = ancho / 2$) o crear patrones matemáticos, elevando el nivel de abstracción por encima del SVG estático.

Cabe destacar que la lógica que habilita esta funcionalidad fue desarrollada íntegramente en esta etapa. Se extendió y adaptó el módulo Calculator (preexistente en la arquitectura base) para actuar durante la fase de análisis semántico, realizando *Constant Folding*. Esto significa que el compilador resuelve todas las operaciones aritméticas y referencias a variables en tiempo de compilación, generando un SVG final optimizado que contiene únicamente los valores numéricos resultantes, liberando al navegador de cualquier carga de cálculo.

5. Implementación

El compilador ha sido desarrollado en lenguaje C, priorizando el rendimiento y la gestión eficiente de memoria. La arquitectura sigue el diseño clásico de dos etapas: Frontend y Backend.

5.1. Frontend: Análisis Léxico y Sintáctico

El frontend transforma el código fuente en una representación estructurada en memoria. Se utilizó la herramienta **Flex** para generar el escáner léxico. El lexer (*FlexPatterns.l*) es responsable de:

- **Tokenizar** palabras clave, identificadores y literales.
- **Manejar** diferentes contextos (Start Conditions) para procesar cadenas de texto y comentarios (línea y bloque) sin ambigüedades.
- **Reconocer** patrones complejos como colores hexadecimales y unidades de medida, adjuntando esta información al valor semántico del token.

El parser fue generado con **Bison** (*BisonGrammar.y*). Se diseñó una Gramática Libre de Contexto (CFG) que define la estructura jerárquica del lenguaje.

- **Construcción del AST:** Mediante acciones semánticas en C, el parser construye progresivamente un Árbol de Sintaxis Abstracta (AST). Los nodos del AST (Scene, Layer, Figure, Property) son estructuras (struct) en C enlazadas dinámicamente.
- **Manejo de Errores:** Se implementó un mecanismo de recuperación de errores básicos y reporte de ubicación (línea y columna) para facilitar la depuración al usuario.
- Como **mejora significativa** respecto a etapas anteriores, se eliminó el parseo manual de cadenas (uso de `sscanf` en C) para estructuras complejas como coordenadas, colores funcionales (rgb, rgba) y dimensiones. En su lugar, se optó por un parseo estructural definido puramente en la gramática de Bison. Esto no sólo simplificó el código C, eliminando deuda técnica, sino que delegó la validación sintáctica a la herramienta generadora, resultando en una detección de errores más robusta y un mantenimiento más sencillo.

5.2. Backend: Semántica y Generación

El backend recibe el AST crudo y realiza las validaciones lógicas y la traducción final. El módulo SemanticAnalyzer realiza múltiples pasadas sobre el AST para asegurar la correctitud del programa:

- **Tabla de Símbolos:** Se implementó una estructura de datos robusta para gestionar los ámbitos (scopes) de variables, paletas y símbolos.
- **Resolución de Referencias:** Se valida que todo color referenciado (ej. fill paleta.rojo) y todo símbolo instanciado (ej. use arbol) hayan sido definidos previamente.
- **Validación de Tipos y Rangos:** Se verifican restricciones del dominio, como que la opacidad esté en el rango $[0, 1]$, que las dimensiones sean positivas y que los componentes RGB estén en $[0, 255]$.
- **Unicidad:** Se detectan colisiones de identificadores y, crucialmente, colisiones de z-index en las capas, garantizando un orden de renderizado determinista.

El módulo Calculator es el motor matemático del compilador. Recorre los sub-árboles de expresiones dentro del AST y las reduce a valores constantes.

- Soporta operaciones aritméticas básicas: suma, resta, multiplicación y división.
- Resuelve referencias a variables almacenadas en la tabla de símbolos.
- Permite que propiedades como width se definan como $100 * 2 + \text{margen}$, calculando 210 (asumiendo margen=10) antes de generar el código.

El módulo Generator es la etapa final. Transforma el AST validado y simplificado en XML.

- **Algoritmo de Renderizado por Capas:** SVG dibuja en orden de aparición (el último elemento tapa al primero). Plotter permite declarar capas en cualquier orden. El generador recolecta todas las figuras de todas las capas, las ordena según su atributo z-index y luego emite el código SVG en ese orden ordenado. Esto desacopla la definición lógica de la visual.
- **Traducción de Propiedades:** Convierte las estructuras internas de C en atributos SVG estándar (ej. fill="#FF0000", transform="rotate(45)"), respetando las unidades de medida originales del usuario.

5.3. Infraestructura y Calidad

El proyecto incluye una infraestructura de desarrollo profesional:

- **Dockerización:** Un contenedor Docker encapsula todas las dependencias (GCC, Flex, Bison, CMake), asegurando que el entorno de compilación sea idéntico para todos los desarrolladores.
- **Gestión de Recursos:** Se implementó una política más clara de propiedad de recursos, especialmente para cadenas de texto e identificadores en el AST. Además, se agregaron destructores recursivos para los nodos del árbol y la tabla de símbolos, mejorando significativamente la gestión de los objetos durante la compilación.
- **Testing Automatizado:** Un script de bash (test.sh) ejecuta una batería de más de 30 pruebas de integración. Estas pruebas se dividen en:

- Accept Tests: Programas válidos que deben compilar y generar un SVG correcto.
- Reject Tests: Programas con errores sintácticos o semánticos que el compilador debe rechazar con un mensaje de error adecuado.
- **Evolución de los Tests:** La transición al Stage III implicó una reorganización y expansión significativa de los casos de prueba para reflejar la capacidad completa del compilador (Frontend + Backend)
 - Reclasificación de Tests: Varios tests que en el Stage II se consideraban *reject* debido a la falta de implementación del backend (por ejemplo, aquellos con expresiones matemáticas complejas como `18-math-expressions`), fueron movidos a *accept* una vez que el módulo *Calculator* y el *Generator* estuvieron operativos.
 - Nuevos Casos de Prueba: Se agregaron tests específicos para validar las nuevas funcionalidades semánticas, como: `19-complex-math-variables` y `20-advanced-variables`, que verifican la correcta resolución de scopes y precedencia de operadores.
 - Refinamiento Semántico: Se eliminaron tests que rechazaban dimensiones sin unidades (ej. `width 50`), ya que se implementó una regla de inferencia que asigna `px` por defecto, mejorando la experiencia de usuario. Se endureció la validación semántica para evitar ambigüedades. Por ejemplo, el test `10-incomplete-property` (en *reject*) verifica que no sea posible definir una figura con propiedades contradictorias o insuficientes, como declarar `width` y `height` junto con `size` (lo cual duplicaría la definición), o declarar sólo `width` en un rectángulo sin especificar `height`. Ahora el compilador exige explícitamente pares completos (`width+height`) o la propiedad taquigráfica `size`.

5.4 Dificultades encontradas

Durante el ciclo de desarrollo, el equipo superó varios desafíos técnicos significativos:

5.4.1 Integración de Expresiones Matemáticas en la Gramática

Integrar expresiones aritméticas dentro de las propiedades de las figuras (ej. `width 50 + 10`) generó conflictos iniciales en la gramática.

El problema fueron las ambigüedades entre el uso de paréntesis para agrupar expresiones matemáticas y el uso de paréntesis para definir coordenadas (x, y). Como solución se refactorizó la gramática para distinguir claramente entre un token `COORDINATES` (tupla) y una expresión aritmética general, y se ajustó la precedencia de operadores en Bison para asegurar una evaluación correcta.

5.4.2 Ordenamiento Visual vs. Declarativo

SVG no posee una propiedad `z-index` nativa como CSS; el orden de dibujado es estrictamente secuencial.

Como solución, implementamos un algoritmo en el backend que "aplana" la estructura jerárquica del AST antes de la generación. Se extraen todos los elementos renderizables, se ordenan mediante un algoritmo de ordenamiento estable (`stable sort`) basado en el `z-index` de su capa contenedora, y finalmente se emiten al archivo de salida. Esto permite al usuario pensar en "profundidad" en lugar de "secuencia".

5.4.3 Refactorización y Evolución desde el Stage II

Tras recibir la retroalimentación del Stage II, y con el desarrollo del Stage III ya avanzado, el equipo dedicó un esfuerzo a integrar las correcciones sugeridas directamente sobre la versión final del compilador.

Lejos de ser un obstáculo, este proceso de refactorización sirvió para consolidar la robustez del proyecto antes de la entrega final:

- **Unificación del Parseo:** Al reemplazar la lógica manual por reglas gramaticales formales en el Frontend, logramos que el Backend (que ya estaba implementado) recibiera datos más confiables, eliminando posibles casos bordes que no habíamos detectado inicialmente.
- **Gestión de Recursos:** La implementación de destructores y la corrección en el manejo de strings en la Tabla de Símbolos fueron fundamentales para soportar la complejidad agregada por el manejo de variables y scopes en esta etapa, evitando que el consumo de memoria crezca descontroladamente. Los nuevos módulos (Calculator, Semantic Analyzer) no introdujera regresiones en el manejo de recursos.
- **Calidad de Código:** Se mejoró la calidad general del código con el objetivo de fortalecer la mantenibilidad del proyecto. Se eliminaron magic numbers las cuales fueron reemplazados por constantes más claras, se reemplazó el abort() por un manejo de errores controlado y por último se renombraron variables y funciones confusas para mejorar la legibilidad.

De esta forma, la entrega final no solo cumple con los requisitos del Stage III, sino que eleva el estándar de calidad de todo el código base.

6. Futuras Extensiones

Aunque **Plotter** es funcional y potente, su arquitectura modular permite expansiones interesantes:

1. **Animaciones y Dinamismo:** Incorporar soporte para animaciones SVG (SMIL) o inyección de CSS keyframes. Esto permitiría definir atributos dinámicos como animate(from, to, duration) directamente en el DSL.
2. **Soporte Tipográfico Completo:** Agregar la primitiva text con soporte para fuentes externas, alineación y renderizado de texto sobre trayectorias (text-on-path), ampliando el uso de la herramienta para infografías.
3. **Gráficos Estadísticos de Alto Nivel:** Crear abstracciones sobre las primitivas actuales para generar gráficos de barras, líneas o torta automáticamente a partir de arrays de datos, convirtiendo al lenguaje en una herramienta de Data Visualization.
4. **Backend de Optimización:** Implementar una fase de post-procesamiento en el generador que optimice el SVG resultante (minificación, reducción de precisión decimal innecesaria, uso de <path> compactos en lugar de polígonos verbosos) para reducir el tamaño del archivo web.

7. Conclusiones

El desarrollo de **Plotter** permitió materializar un lenguaje de dominio específico para la gráfica vectorial 2D que abstrae la complejidad de SVG mediante una sintaxis declarativa de alto nivel. A partir de conceptos como escenas, capas con z-index explícito, grupos, símbolos reutilizables, paletas de colores semánticas y un sistema de unidades flexible, el lenguaje ofrece al usuario una forma más natural de describir composiciones visuales complejas, separando la definición lógica de los detalles de implementación propios del formato de salida.

Desde el punto de vista de la implementación, se logró construir un compilador completo en C organizado en etapas bien definidas. El frontend, basado en Flex y Bison, transforma el código fuente en un AST estructurado, mientras que el backend, compuesto por un analizador semántico, una tabla de símbolos, un módulo de cálculo de expresiones y un generador de código, valida las restricciones del dominio y produce SVG válido respetando el orden de renderizado por capas. La incorporación de dockerización y de una batería de pruebas de aceptación y rechazo contribuyó a asegurar la reproducibilidad del entorno y la calidad de la solución durante todo el ciclo de desarrollo.

El proceso de diseño e implementación evidenció la importancia de contar con una gramática clara, de tratar cuidadosamente las ambigüedades entre sintaxis declarativa y expresiones aritméticas, y de desacoplar las decisiones de modelado lógico de las particularidades del backend de renderizado. Las soluciones adoptadas frente a problemas como el manejo de expresiones en propiedades y el ordenamiento visual de elementos sientan una base sólida para extender **Plotter** con nuevas capacidades (animaciones, soporte tipográfico avanzado o primitivas de visualización de datos) y constituyen un aporte formativo relevante en la construcción de lenguajes y compiladores orientados a dominios específicos.

8. Referencias

- Scalable Vector Graphics (SVG) 1.1 (Second Edition). W3C Recommendation 16 August 2011. <https://www.w3.org/TR/SVG11/>
- Flex - The Fast Lexical Analyzer. Documentación oficial del proyecto GNU.
- Bison - The GNU Parser Generator. Free Software Foundation.

9. Bibliografía